

Réagir automatiquement aux actions de Claude Code

Qu'est-ce qu'un hook

Un hook est un script shell (ou toute commande) exécuté automatiquement quand Claude Code déclenche un événement. Comparable aux git hooks, mais pour l'environnement de l'agent. Le script reçoit un JSON via stdin décrivant le contexte de l'événement.

Événements principaux

Événement	Moment	Peut bloquer
SessionStart	Démarrage ou reprise	Non
SessionEnd	Fin de session	Non
UserPromptSubmit	Avant traitement du prompt	Oui
PreToolUse	Avant l'appel d'un outil	Oui
PostToolUse	Après succès d'un outil	Non
PreCompact	Avant compaction du contexte	Oui
Notification	Claude envoie une notification	Non
Stop	Claude finit de répondre	Oui
SubagentStop	Fin d'un sous-agent	Oui

Événements Agent Teams (v2.1.32+)

Événement	Utilité
TeamateIdle	Un membre de l'équipe va passer en idle
TaskCompleted	Une tâche est sur le point d'être marquée terminée
ConfigChange	Un fichier de config a changé pendant la session

Ces événements permettent d'implémenter des quality gates au niveau de l'équipe d'agents : empêcher qu'une tâche soit marquée `completed` si les tests échouent, par exemple.

Configuration dans settings.json

```
{  "hooks": {    "PostToolUse": [      {        "matcher": "Edit|Write",        "hooks": [          {            "type": "command",            "command": ".claude/hooks/auto-format.sh",            "timeout": 10000,            "async": true          }        ]      },      {        "matcher": "Bash",        "hooks": [          {            "type": "command",            "command": ".claude/hooks/security.sh"          }        ]      }    ]  }
```

Champs de configuration

Champ	Usage
command	Commande shell exécutée via interpréteur
args	Forme exec : tableau de chaînes spawné directement sans shell. Évite les risques d'injection et les problèmes de quoting. Remplace

Champ	Usage
	<code>command</code> quand présent. (v2.1.139)
<code>timeout</code>	Délai max en ms (défaut : 60000)
<code>async</code>	Si <code>true</code> , Claude continue sans attendre la fin
<code>continueOnBlock</code>	<code>PostToolUse</code> uniquement : si <code>true</code> , un blocage alimente Claude avec la raison du refus et continue le tour au lieu de l'arrêter. Claude peut alors s'autocorriger. (v2.1.139)

Codes de sortie

Code	Signification
0	Succès, continuer normalement
2	Bloquer l'action (message stderr visible par Claude)
Autre non-0	Erreur remontée à Claude

Pour les hooks `PreToolUse`, un exit 2 empêche l'outil de s'exécuter. Claude lit le message stderr et peut adapter son comportement.

Synchrone vs Asynchrone

Hooks synchrones (défaut) : Claude attend la fin du script. Adapté à la validation et au type checking. Hooks asynchrones (`async: true`) : Claude continue immédiatement. Adapté au formatage, aux notifications et aux logs, où le feedback n'est pas nécessaire pour la suite.

Flux des données stdin

```
# Exemple de JSON reçu sur stdin (PostToolUse) {  "tool_name": "Edit",  "tool_input": {    "file_path": "src/auth.ts"  },  "tool_response": {    "success": true  },  "hook_event_name": "PostToolUse" }
```

L'événement `Stop` inclut également un champ `last_assistant_message` depuis v2.1.47, donnant accès direct à la dernière réponse de Claude sans parser les transcripts.